

Prediction Engine Module

Documentation

Overview

This Python module implements an enhanced predictive engine designed to anticipate a player's input (numbers 1-10) in a game like Hand Cricket. The engine leverages historical data stored in CSV files and employs increasingly sophisticated, data-driven strategies across different difficulty levels. Key enhancements include proportional grading of prediction accuracy and a dynamic approach to selecting numbers from historical data based on recent performance.

Core Concepts

1. **Data Persistence:** The engine distinguishes between:
 - **Instance Data (i_):** Data collected during the current game session. This data can be reset under specific conditions based on difficulty.
 - **Permanent Data (p_):** Data collected across all game sessions, providing long-term trends. This data persists across game sessions.
2. **Data Categories:** The engine tracks four main types of data:
 - **UCR (User-Computer Record):** Logs pairs of player inputs and corresponding computer predictions for each round.
 - **Heatmap:** Tracks statistics for each number (1-10) based on how often it was *predicted*, and the outcome of those predictions. The **Accuracy** score is now calculated **proportionally** to the prediction error. Used to evaluate prediction performance per number and contribute to overall performance metrics.
 - **Pattern:** Records sequences of *compressed* player inputs (1-10 mapped to 4 broader categories). Used to identify recurring player behaviors.
 - **ComDistro (Combo Distributive):** Tracks the frequency of specific 3-element compressed patterns derived from the Pattern data.
3. **Proportional Accuracy:** A new method of calculating the Heatmap 'Accuracy' score update. Instead of fixed values, it's calculated as $\text{CorrectBonus} - \text{abs}(\text{playerInput} - \text{comInput})$. Being exactly correct provides CorrectBonus, while being further off results in a larger penalty (more negative update). This makes the Accuracy scores a more granular reflection of prediction quality. (Note: CorrectBonus is implicitly set by the accuracy update logic).

4. **Dynamic UCR Strategy:** At higher difficulties, the strategy for picking from UCR history adapts based on the AI's recent overall performance (netAccuracy). Instead of picking from a fixed window, the logic for selecting a historical number becomes more sensitive to whether the AI's predictions are generally succeeding or failing.
5. **Difficulty Levels:** The engine operates at three difficulty levels, controlling the prediction logic, data usage (instance vs. permanent), and instance data resets:
 - **Difficulty 1:** Basic prediction using random selection from UCR history (instance or permanent).
 - **Difficulty 2:** Uses a mix of UCR history (permanent, with dynamic range based on performance) and weighted selection based on Heatmap performance (instance or permanent, using proportional accuracy). Instance data resets after a fixed number of rounds (12 UCR entries).
 - **Difficulty 3:** Adaptive prediction alternating between a UCR-based and a Pattern-based strategy. The strategy switch is triggered by the overall performance (netAccuracy calculated using proportional grading). Instance Heatmap data is reset when a switch occurs.
6. **Data Compression:** Player inputs (1-10) are compressed into 4 broader categories for pattern analysis (1-3 -> 1, 4-5 -> 2, 6-7 -> 3, 8-10 -> 4). Patterns are further compressed from 6 elements to 3 by averaging pairs.

Global Variables

- **difficulty (int):** The current difficulty level (1, 2, or 3). Default: 2.
- **pattern (list[int]):** A temporary list storing the current sequence of compressed player inputs (up to 6 elements). Cleared after a full 6-element pattern is recorded.
- **style (list):** Declared but currently unused in the provided code.
- **netAccuracy (int):** Stores the sum of the "Accuracy" scores from the instance Heatmap (iHeatmap). Calculated using proportional grading, this provides a more sensitive gauge of the performance of the current prediction method or overall instance performance.
- **method (int):** Counter used in Difficulty 3 to track and alternate between the UCR and Pattern prediction methods. Even values typically indicate UCR, odd values indicate Pattern.

Data Objects

Instances of a custom Data class (aliased as d), responsible for interacting with the CSV datastores. Initialization includes basic error handling for file loading.

- iComDistro: Instance Combo Distributive data (datastorecsv\I_ComboDistributive.csv).
- iHeatmap: Instance Heatmap data (datastorecsv\Instance_Heatmap.csv).
- iPattern: Instance Pattern data (datastorecsv\Instance_Pattern.csv).
- iUCR: Instance UCR data (datastorecsv\Instance_UCR.csv).
- pComDistro: Permanent Combo Distributive data (datastorecsv\P_ComboDistributive.csv).
- pHeatmap: Permanent Heatmap data (datastorecsv\Permanent_Heatmap.csv).
- pUCR: Permanent UCR data (datastorecsv\Permanent_UCR.csv).

Functions

1. 'initialize(inDiff)'

- **Purpose:** Sets the global difficulty level for the prediction engine.
- **Parameters:**
 1. inDiff (int or str): The desired difficulty level.
- **Logic:**
 1. Attempts to convert inDiff to an integer.
 2. If conversion is successful and the integer is 1, 2, or 3, updates the global difficulty variable.
 3. Prints a message indicating the set difficulty or a warning if the input is invalid, in which case the default difficulty (2) remains.

2. 'store_choice(playerInput, comInput, Ball)'

- **Purpose:** Processes the results of a single round (or "ball") and updates the instance and permanent data structures based on the player's choice and the computer's prediction, using the new proportional accuracy grading.
- **Parameters:**
 1. playerInput (int or str): The number chosen by the player (1-10).
 2. comInput (int or str): The number predicted by the computer (1-10).
 3. Ball (float or int): An identifier for the round.
- **Logic:**
 1. **Input Validation:** Converts playerInput and comInput to integers and checks if they are within the 1-10 range. Exits the function with a warning if inputs are invalid.
 2. **Difficulty 2 Reset Check:** If difficulty is 2 and the number of entries in the instance UCR (iUCR) is 12 or more, all instance data objects (iUCR, iHeatmap, iPattern, iComDistro) are reset using their .reset() method. This happens *before* processing the current round's data.
 3. **Update Data Structures:** Calls three nested helper functions to update the data based on the current, validated inputs:
 - **heatmap() (Nested):**
 - Retrieves current heatmap counts/scores for comInput from iHeatmap and pHeatmap.
 - Calculates the absolute difference between playerInput and comInput.
 - Determines the **Accuracy update** using the formula: CorrectBonus - absolute_difference. (Note: The value of CorrectBonus is implicitly 2 in the current code, resulting in updates like 2 - 0 = +2, 2 - 1 = +1, 2 - 2 = 0, 2 - 9 = -7, etc.).
 - Determines Correct (1 if difference is 0, else 0) and Closeby (1 if difference is 1, else 0).

- Updates the "Predictions" count, adds the Correct value to the "Correct" count, adds the Closeby value to the "Closeby" count, and **adds the calculated proportional Accuracy update** to the total "Accuracy" value for comInput in *both* iHeatmap and pHeatmap.

- **UCR() (Nested):**
 - Adds the pair [playerInput, comInput] as a new row to *both* iUCR and pUCR.

- **patternLogic() (Nested):**
 - Uses the nested compression() helper to convert playerInput (1-10) into a compressed category (1-4).
 - Appends the compressed value to the global pattern list.
 - If pattern reaches a length of 6:
 - Adds the full 6-element pattern to iPattern and pPattern.
 - Uses the nested compressMore() helper to create a 3-element list by averaging pairs from the 6-element pattern.
 - Formats the 3-element pattern into a lookup key (e.g., '1,2,3').
 - Searches for this key in iComDistro.
 - If the key is found, increments the "Count" for that pattern in *both* iComDistro and pComDistro.
 - Resets the global pattern list to empty [].
 - Includes a safeguard to reset pattern if its length somehow exceeds 6.

3. 'predict()' -> int'

- **Purpose:** Calculates and returns the computer's prediction (an integer between 1 and 10) based on the current data, difficulty level, and recent performance metrics (netAccuracy).
- **Returns:** An integer (1-10) representing the predicted player input.
- **Logic:**
 1. **Initialize:** Sets final_prediction to None. Calculates the overall netAccuracy by summing the (proportionally graded) "Accuracy" scores from the instance heatmap (iHeatmap). This netAccuracy is a more sensitive indicator of recent performance.
 2. **get_ucr_history(data_obj) (Helper):** Safely retrieves the 'User' column from a UCR Data object, filters for valid integers (1-10), and returns it as a list. Handles errors and empty data.
 3. **basicPatterns() (Helper):** Checks the last 2 or 3 entries in the *instance* UCR (iUCR) for simple repeating patterns (e.g., 5, 5 -> predict 5) or linear progressions (e.g., 3, 5, 7 -> predict 9). Returns a prediction if a pattern is found, otherwise None.
 4. **unpack(avg_compressed_val) (Helper):** Maps an average compressed value back to a range of player input numbers (1-10) and returns a random number within that range.
 5. **Apply Basic Patterns:** Calls basicPatterns() first. If it returns a prediction, this becomes final_prediction.
 6. **Difficulty-Specific Logic:** If final_prediction is still None, executes logic based on difficulty:
 - **Difficulty 1:** Randomly chooses between using the instance UCR history (iUCR) or the permanent UCR history (pUCR). If the chosen history is not empty, picks a random number *from* that history. (Unaffected by new accuracy/dynamic logic).
 - **Difficulty 2:** Randomly chooses between a UCR-based strategy and a Heatmap-based strategy.
 - **UCR Strategy:** Uses the *permanent* UCR history (pUCR). The logic for selecting a number becomes **dynamic** based on netAccuracy:
 - *If netAccuracy is high (e.g., ≥ 0):* Picks a number from the *most recent* few entries (e.g., last 5) as recent history is performing well.
 - *If netAccuracy is moderate (e.g., < 0 but ≥ -10):* Picks from a slightly wider range of recent entries (e.g., last 10).
 - *If netAccuracy is low (e.g., < -10):* Picks from a broader range of history (e.g., last 20 or 30) or potentially introduces more randomness, assuming recent history isn't predictive.
 - *(Note: Specific thresholds and ranges are implementation details requiring tuning, but the principle is to adjust the history window or selection method based on performance).*

- **Heatmap Strategy:** Uses the **instance** (iHeatmap) or **permanent** (pHeatmap) heatmaps. It creates weighted lists of numbers (1-10) where numbers with higher **proportionally graded** Accuracy scores are more likely to be chosen. Randomly picks between the instance or permanent weighted list and then selects a number based on the weights.
- **Difficulty 3:** Checks the netAccuracy (calculated using proportional grading).
 - If netAccuracy ≥ -10 : Continues using the strategy indicated by the method counter (even: UCR, odd: Pattern).
 - **UCR Method (method even):** Uses the *permanent* UCR history (pUCR) with the **same dynamic range logic as described for Difficulty 2**, adjusting the history window or selection based on netAccuracy.
 - **Pattern Method (method odd):** Uses the *current incomplete* global pattern list to find matching patterns in the **permanent** pPattern data. Averages the compressed 'next' numbers from matching patterns and uses unpack() for the prediction. (Logic unchanged from previous version, but the decision to *use* this method is influenced by the new netAccuracy).
 - If netAccuracy < -10 : Resets the instance heatmap (iHeatmap.reset()) and increments the method counter to switch strategy for the *next* prediction. final_prediction remains None, triggering the fallback.
- 7. **Fallback to Random:** If final_prediction is still None after all difficulty-specific logic, or if the resulting prediction is not a valid integer between 1 and 10, a random number between 1 and 10 is chosen as the final_prediction.
- 8. **Return:** Returns the final_prediction as an integer.

Data Structure Layout (Conceptual)

The conceptual layout remains the same, but the content of the 'Accuracy' column in the Heatmap CSVs now reflects the summed values from proportional updates (CorrectBonus - difference).

- ***_UCR.csv:**
| User | Computer |
| :--- | :----- |
| 5 | 3 |
| 7 | 7 |
| ... | ... |
- ***_Heatmap.csv:**
| Number | Predictions | Accuracy | Correct | Closeby |
| :----- | :----- | :----- | :----- | :----- |
| 1 | X | Y (Proportional Sum) | Z | A |
| 2 | X | Y (Proportional Sum) | Z | A |
| ... | ... | ... | ... | ... |
| 10 | X | Y (Proportional Sum) | Z | A |
- ***_Pattern.csv:** (No change)
| I1 | I2 | I3 | I4 | I5 | I6 |
| :- | :- | :- | :- | :- | :- |
| 1 | 2 | 3 | 1 | 4 | 2 |
| ... | ... | ... | ... | ... | ... |
- ***_ComDistro.csv:** (No change)
| PatternKey | Count |
| :----- | :----- |
| 1,2,3 | X |
| ... | ... |

Module Flow

The sequence of function calls remains the same, but their internal logic is enhanced:

1. Call `initialize()` once to set difficulty.
2. Call `predict()` to get the computer's prediction. This now calculates `netAccuracy` using proportional grading and uses it to inform the dynamic UCR strategy in Diff 2/3 or trigger Diff 3 strategy switches.
3. After the round, call `store_choice()`. This function now updates the Heatmap accuracy using the proportional grading formula. This new accuracy score feeds into the `netAccuracy` calculation in the *next* call to `predict()`. It also continues to update UCR and Pattern data.
4. Repeat steps 2-3.

Instance data is managed: volume-based reset in Difficulty 2 (`store_choice`), and performance-based heatmap reset in Difficulty 3 (`predict` when `netAccuracy` is low).